

Predator_5 Quick Start

8 Steps to Train and Evaluate Proof-Search Policies

Step 1: Open PowerShell

Press **Win + R**, type **powershell**, and press Enter. You will see a prompt like **PS C:\Users\YourName>**

Step 2: Navigate to the outputs folder

```
cd C:\Users\12096\AppData\Roaming\Claude\local-agent-mode-sessions\8db2ec26-c9d2-4bf4-9004-f935cf3
```

Verify you are there: **ls** should list **predator5.py**

Step 3: Check the environment

```
python predator5.py doctor
```

Output shows Python version and whether numpy/sklearn are installed. No action needed if sklearn is missing for step 5; logistic regression works without it.

Step 4: Harvest the ground truth

```
python predator5.py harvest --depth 4 --edge-cap 15 --max-size 14
```

Runs breadth-first sweep. Prints geodesic distances (least m with $w \in p_m$) for 63 targets. Keep the output — you will need the held-out target list for later steps.

Step 5: Train the logistic ranker

```
python predator5.py train --model logistic --depth 4 --seed 0
```

Prints 12 fitted weights. Positive weight: higher score $\rightarrow$ more likely on-geodesic. Negative: the reverse.

Step 6: (Optional) Train the random forest

First, install scikit-learn if you haven't:

```
pip install scikit-learn
```

Then train:

```
python predator5.py train --model forest --depth 4 --seed 0 --n-estimators 300 --max-depth 12 --mi
```

Prints Gini importances for each feature.

Step 7: Compare both models on held-out targets

This is the command that matters. It fits logistic and forest on training targets, then evaluates both in reorder and prune modes against breadth-first search on held-out targets.

```
python predator5.py compare --depth 4 --edge-cap 12 --max-size 14 --budget 150 --lam 0.5 -k 4 --te
```

Output appears both on screen and in **results.json**.

Step 8: Verify stability across random splits

Runs step 7 with five different random seeds. If the speedup swings wildly, the 19-target held-out set is too small.

```
for ($S=0; $S -lt 5; $S++) { python predator5.py compare --depth 4 --seed $S --budget 150 --n-esti
```

Bonus: Sweep the interpolation knob (λ)

At $\lambda=0$ every row must collapse to BFS; if it does not, the depth term is miswired. As λ grows, expansions should fall and then rise as the search starts diving.

```
for L in 0 0.25 0.5 1 2 4 8; do echo "=== lambda = $L ==="; python predator5.py compare --depth 4
```

Bonus: Widen the sweep for more targets

Deeper sweep, more targets, bigger held-out set. Slower — the state count grows fast.

```
python predator5.py compare --depth 5 --edge-cap 12 --max-size 14 --budget 400 --lam 0.5 -k 4 --te
```